

COMP 8003

Lab 5

Report

Louise Li
A01377263
Feb 9th, 2026

Purpose	3
Requirements	3
Platforms	3
Language	3
Documents	3
Findings	3

Purpose

The lab demonstrates the weaponization phase of the cyber kill chain by designing and executing a benign activity that interacts with internal data using regular user-level system behavior.

Requirements

Task	Status
Shell script creates a hidden directory in the user's home directory.	Fully implemented
Shell script copies /etc/passwd into the created hidden directory.	Fully implemented
Use inotifywait to monitor access to /etc/passwd.	Fully implemented
Create a short list of assumptions to what is considered as "normal" within a system.	Fully implemented
Write a short written analysis connecting offensive action to defensive observation.	Fully implemented
Write a reflection from a defensive perspective.	Fully implemented

Platforms

Script has been tested on:

- Ubuntu 24.04.1 LTS

Language

- sh

Documents

- [Design](#)
- [Testing](#)
- [User Guide](#)

Findings

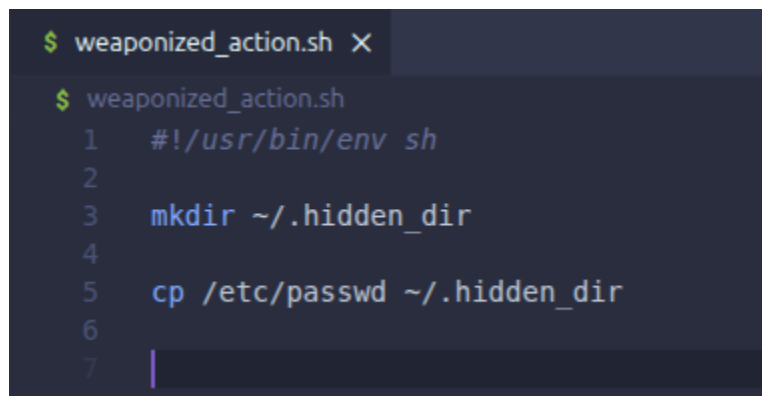
Part 1: Understanding the Environment

List of assumptions about what is considered as “normal” on the system:

- Regular users without root privileges can create, read, modify, delete, and execute files they hold ownership over within user-writable directories.
- Regular users without root privileges can read certain world-readable system files but cannot modify protected or kernel resources
- Hidden directories are often used to store internal data.
- Software often creates directories to store configuration or internal data.
- Copying readable files within user-accessible locations is a common routine.
- Shell scripts and command line utilities are frequently run.

Part 2: Designing a Weaponized Action

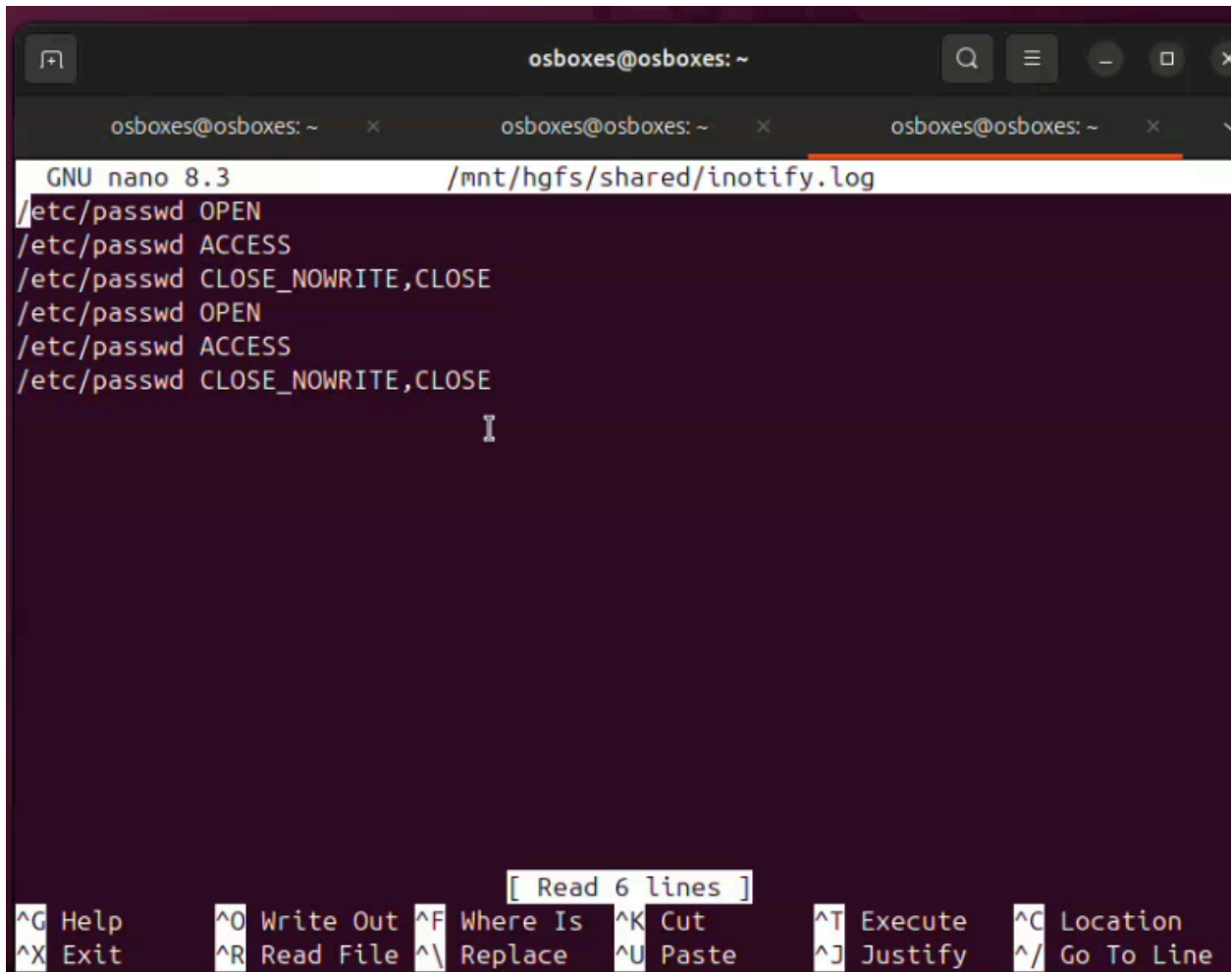
A simple shell script run by sh that creates a hidden directory, *hidden_dir*, in the user’s home directory. The script then copies */etc/passwd* into this directory, making a local copy of the system’s user account list.



```
$ weaponized_action.sh X
$ weaponized_action.sh
1  #!/usr/bin/env sh
2
3  mkdir ~/.hidden_dir
4
5  cp /etc/passwd ~/.hidden_dir
6
7  |
```

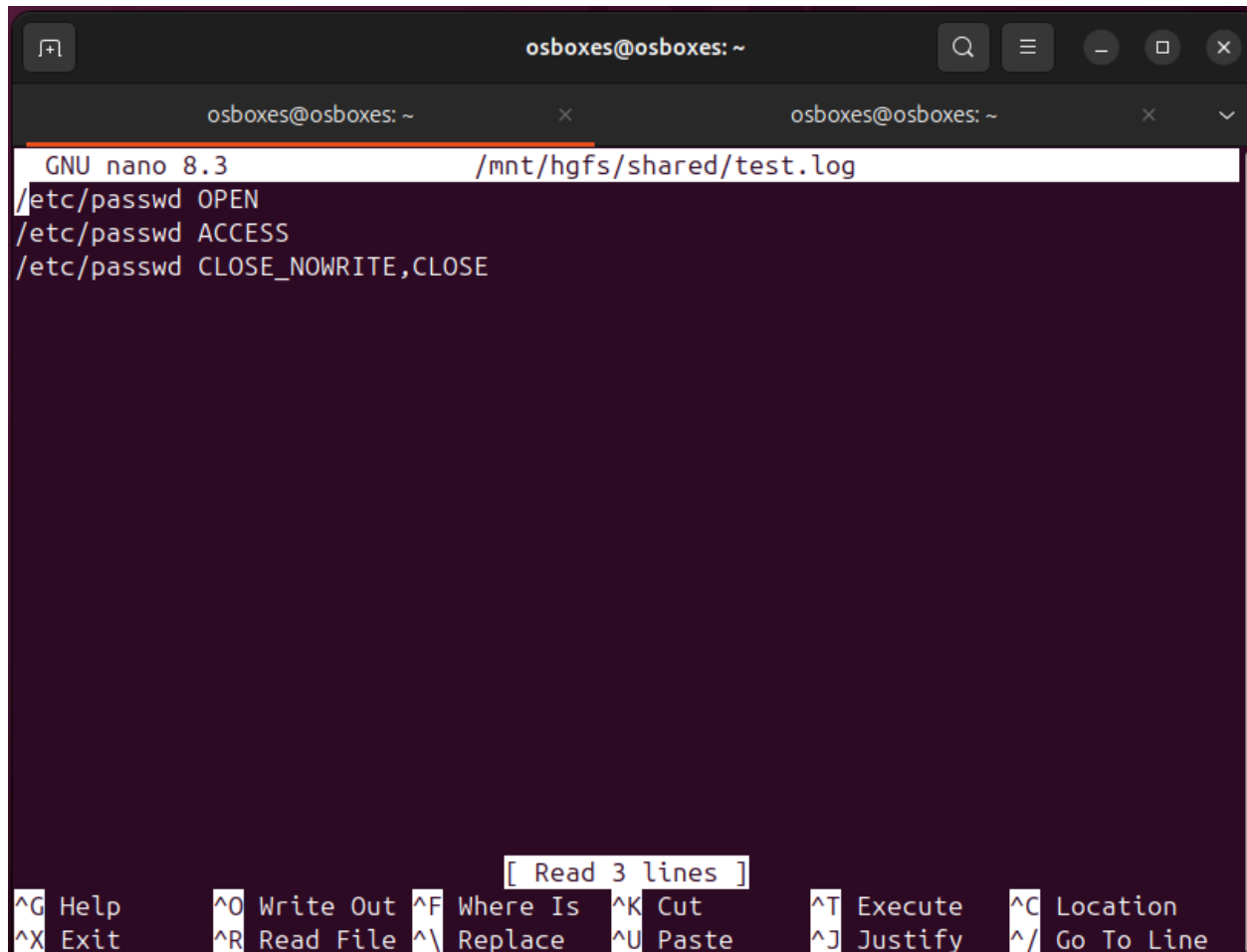
Part 3: Observing Weaponization as a Defender

Attempt 1:



```
osboxes@osboxes: ~  
GNU nano 8.3 /mnt/hgfs/shared/inotify.log  
/etc/passwd OPEN  
/etc/passwd ACCESS  
/etc/passwd CLOSE_NOWRITE,CLOSE  
/etc/passwd OPEN  
/etc/passwd ACCESS  
/etc/passwd CLOSE_NOWRITE,CLOSE  
I  
[ Read 6 lines ]  
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

Attempt 2:



```
osboxes@osboxes: ~  
GNU nano 8.3 /mnt/hgfs/shared/test.log  
/etc/passwd OPEN  
/etc/passwd ACCESS  
/etc/passwd CLOSE_NOWRITE,CLOSE  
[ Read 3 lines ]  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Part 4: Correlating Actions and Artifacts

The defender can observe the access, read, and close events of the file, as well as indicating whether any write activity occurred. These actions could be seen as benign because no files were modified and all operations are allowed for a normal user. However, copying `/etc/passwd` creates a locally accessible snapshot that could support later activity if combined with additional actions.

Timing affects interpretation because file access occurs immediately during script execution, which resembles regular user behavior. Without additional context, the activity does not appear to be malicious. The artifacts left behind include the `/etc/passwd` access log entries generated and the hidden directory containing the copied file.

Part 5: Defender Reflection

Access to `/etc/passwd` is normal because the file is world-readable, making individual read events difficult to classify as malicious. Additional context like system baselines and patterns of user behavior would increase confidence when interpreting the activity. Host-based intrusion detection or auditing mechanisms that can correlate file access with other activity like process

execution and timing would make the behavior easier to detect and analyze. On its own, the activity does not generate enough reason to be classified as malicious to justify escalation, as it does not cause harm nor breach any policies. Escalation would be justified if the activity was paired with additional behavior that breached policies or exceeded regular user activity.